

Ticket Routing Agent -- Design Spec

Date: 2026-04-10
Status: Draft
Context: Vodafone Global Service Desk / TechMahindra engagement

Problem Statement

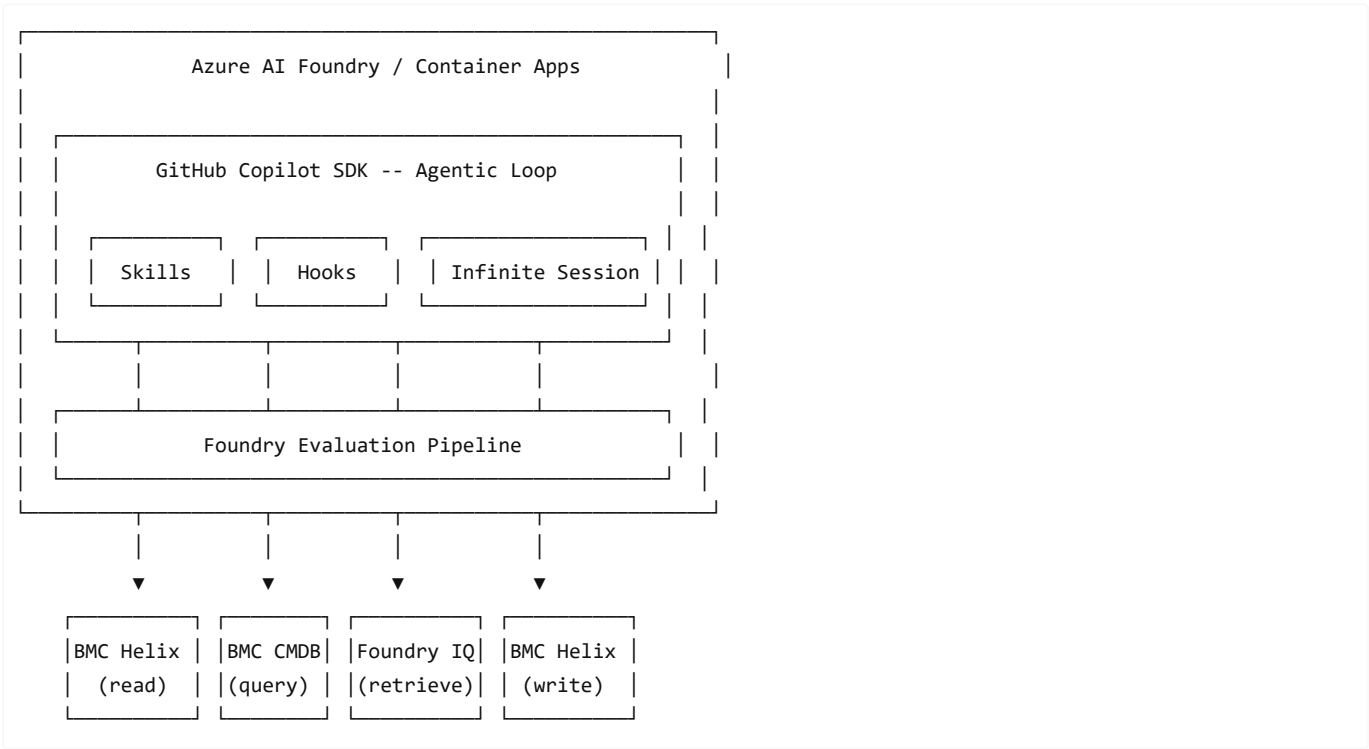
Vodafone's Global Service Desk handles ~16,000 P3/P4 incident tickets per month. L1 engineers manually inspect BMC CMDB for CI relationships and ownership, then route to one of ~2,500 resolver groups. Tickets frequently bounce between groups before reaching the correct owner.

Previous ML approaches (2017-present) achieved 65-70% accuracy by training classifiers on ticket text alone, without CMDB dependency and ownership context. None reached production.

Target: >95% first-time-right assignment accuracy.

Architecture

Single GitHub Copilot SDK agentic loop with skill-based architecture, deployed in Azure AI Foundry or Container Apps.



Skills

Skills encapsulate discrete routing capabilities. Each skill owns a bounded responsibility and can be tested, evaluated, and versioned independently.

Skill	Responsibility
ticket-intake	Read incident from BMC Helix. Extract CI, category, priority, symptom description. Normalize and structure the payload for downstream skills.
cmdb-lookup	Query BMC CMDB for the CI's dependency tree and resolver group ownership. Return structured relationship data.

historical-match	Query Foundry IQ for similar correctly-resolved historical tickets. Return top-N matches with resolver groups and confidence scores.
route-decision	Consume outputs from cmdb-lookup and historical-match. Select resolver group. Produce confidence score and structured reasoning.
ticket-assign	Write resolver group assignment back to BMC Helix. Attach agent reasoning to ticket notes.
data-quality-flag	When CMDDB returns missing, conflicting, or ambiguous data, log the gap for remediation and route ticket to human review.

Skill Orchestration

The agentic loop sequences skills based on context:

1. `ticket-intake` -- always runs first
2. `cmdb-lookup` -- runs on extracted CI
3. `historical-match` -- runs in parallel with `cmdb-lookup` for ambiguous cases, or as fallback if CMDDB data is incomplete
4. `route-decision` -- consumes upstream outputs, selects resolver group
5. `ticket-assign` -- executes if confidence exceeds threshold
6. `data-quality-flag` -- triggers when upstream skills detect data gaps

Hooks

Hooks provide validation gates at defined points in the agent loop. They execute synchronously before the agent proceeds.

Hook	Trigger	Behavior
pre-assign	Before <code>ticket-assign</code> writes to BMC Helix	Validate: resolver group exists in canonical list, ticket is still in assignable state, confidence meets threshold. Block assignment if validation fails.
post-intake	After <code>ticket-intake</code> extracts ticket data	Validate: required fields present (CI, description, category). If incomplete, route to <code>data-quality-flag</code> instead of proceeding.
confidence-gate	After <code>route-decision</code> produces a score	Enforce threshold policy: >0.9 auto-assign, 0.7-0.9 assign with human flag, <0.7 route to L1 queue with top-3 suggestions. Thresholds are configurable.
post-assign	After <code>ticket-assign</code> completes	Log assignment event for Foundry evaluation pipeline. Capture: ticket ID, selected resolver group, confidence, reasoning, timestamp.

Infinite Sessions

The agent runs as a persistent, long-lived session rather than per-ticket ephemeral invocations. Benefits:

- **Accumulated context:** The agent retains awareness of recent routing decisions, emerging patterns, and CMDDB state across tickets within a session window.
- **Batch reasoning:** Related tickets arriving in sequence (e.g., during an outage) can be correlated and routed as a group.
- **Session continuity:** State persists across ticket events. The agent can revisit earlier decisions if new information arrives (e.g., a ticket update that changes the CI).

Session lifecycle: the agent runs continuously, consuming ticket events from a queue. Sessions are bounded by a configurable time window or token budget, then gracefully restart with summary context carried forward.

Foundry IQ (Historical Retrieval)

Foundry IQ replaces a standalone search index. It provides agentic retrieval with built-in grounding over the historical ticket corpus.

Data pipeline:

- Export 12-24 months of resolved P3/P4 tickets from BMC Helix
- Use final resolver group as ground truth label (accounts for reassignments)
- Index: ticket description, metadata, CI, final resolver group
- Periodic refresh (weekly)

Query pattern: The `historical-match` skill queries Foundry IQ with the current ticket context. Foundry IQ returns semantically similar resolved tickets with their routing outcomes, grounded in the indexed corpus.

Foundry Evaluation and Drift Monitoring

Azure AI Foundry evaluation pipeline provides continuous quality monitoring.

Evaluation dataset: Maintained from production traffic -- each assigned ticket becomes an evaluation record once its routing outcome is confirmed (resolved without reassignment = correct, reassigned = incorrect).

Metrics tracked:

- First-time-right rate (overall and per business function)
- Confidence calibration (does 0.9 confidence actually mean 90% correct?)
- Reassignment rate by resolver group
- CMDB data gap frequency

Drift detection: Foundry evaluation runs on a scheduled cadence against the rolling evaluation dataset. Alerts trigger when:

- First-time-right rate drops below threshold
- Confidence calibration degrades (model overconfidence/underconfidence)
- New resolver groups appear that the agent has not seen
- CMDB data gap rate increases

Response: Drift alerts feed into the agent's configuration. Confidence thresholds can be tightened automatically. Persistent drift triggers a review of the historical index freshness and CMDB data quality.

Data Quality

Known issues from customer documentation:

- Resolver group name hygiene (duplicate/stale group names)
- Incorrect CI-to-group mappings
- ~70% of tickets arrive with incomplete information

The `data-quality-flag` skill logs specific gaps as the agent encounters them: missing CI ownership, conflicting resolver groups, unmapped CIs. These logs aggregate into a remediation queue for the CMDB team. The Foundry evaluation pipeline tracks data gap frequency as a metric.

MVP Scope

Scope to one business function (e.g., Cloud & Infrastructure -- 11K of 16K tickets):

1. **Read-only mode:** Agent classifies but does not assign. Compare agent output against actual routing for 2 weeks to establish baseline accuracy.
2. **Shadow mode:** Agent assigns but L1 can override. Measure first-time-right rate.
3. **Autonomous mode:** Agent assigns directly for tickets above confidence threshold.

Success Criteria

- *95% first-time-right assignment on auto-assigned tickets*
- Reduction in average ticket hops from current baseline
- Reduction in MTTR for routed tickets

- Measurable CMDB data quality gap detection rate

Open Questions

1. BMC Helix API access: REST version, authentication, rate limits?
2. CMDB query interface: programmatic access to CI relationships and ownership?
3. Resolver group structure: canonical list with ownership metadata?
4. Typical ticket payload: fields, free-text quality, CI linkage?
5. Event mechanism for new tickets: webhook or polling?
6. Current reassignment rate (baseline)?
7. Sample export availability: 1,000 resolved tickets with routing history?